

Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

DDD: entendendo o domínio do problema

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

Análise Crítica do DDD

Sprint 5 — Análise Crítica (12/06/2026) • Vantagens, limitações, desafios e comparação com outra abordagem de modelagem

O Domain-Driven Design (DDD) propõe modelar o software a partir de um entendimento profundo do domínio de negócio, usando a Linguagem Ubíqua e padrões táticos como Entidades, Value Objects, Agregados e Repositórios. Depois de aplicarmos esses conceitos ao domínio do Scrunic — Tarefa, Quadro, Coluna e Usuário — esta análise examina criticamente seus pontos fortes, suas limitações e seus desafios, e o compara com a modelagem anêmica tradicional baseada em CRUD.

1. Vantagens

Na prática do nosso projeto, o DDD trouxe ganhos claros de clareza e manutenção:

Modelo fiel ao negócio. Nomear classes e tabelas como Tarefa, Quadro e Coluna, em vez de nomes genéricos, tornou o código mais fácil de entender para qualquer novo integrante.

Regras de negócio protegidas. Concentrar as invariantes na Raiz de Agregação (o Quadro) evitou que uma alteração indevida em uma Coluna quebrasse o estado consistente do sistema.

Comunicação melhorada. A Linguagem Ubíqua eliminou traduções erradas entre o que a equipe discutia nas reuniões e o que aparecia no código.

Fronteiras claras. O Contexto Delimitado ajudou a equipe a decidir onde termina a responsabilidade do módulo de tarefas e começa a de usuários.

2. Limitações

Apesar dos ganhos, o DDD também mostrou limitações no nosso contexto:

Curva de aprendizado alta. Distinguir Entidade de Value Object e identificar a Raiz de Agregação corretamente exigiu tempo de estudo antes da equipe ganhar confiança.

Overhead para domínios simples. Para um CRUD simples como o cadastro de usuários, criar Repositório e Agregado completos pareceu esforço desproporcional ao problema.

Depende de acesso a especialistas de negócio. Sem alguém que conheça bem o domínio real, a Linguagem Ubíqua vira apenas um vocabulário interno da equipe técnica.

Risco de modelagem prematura. Definir Agregados cedo demais, antes de entender bem o domínio, pode travar decisões de design que seriam melhores mais adiante.

3. Desafios de adoção

No Scrunic, os principais desafios ao adotar o DDD foram:

Decidir o que é Entidade e o que é Value Object. A prioridade de uma tarefa, por exemplo, gerou discussão até a equipe perceber que ela não tem identidade própria.

Escolher a Raiz de Agregação certa. Definir o Quadro, e não a Coluna ou o Cartão, como responsável por garantir a consistência do conjunto exigiu algumas iterações.

Manter a Linguagem Ubíqua viva. Garantir que os termos usados nas reuniões continuassem batendo com os nomes no código exigiu revisão constante.

4. Comparação com outra abordagem de modelagem: modelagem anêmica (CRUD tradicional)

Comparamos o DDD com a modelagem anêmica — comum em sistemas CRUD tradicionais, em que classes armazenam apenas dados e a lógica de negócio fica isolada em serviços externos.

Critério	DDD (modelo rico)	Modelagem anêmica (CRUD tradicional)
Localização da regra de negócio	Dentro das Entidades e Agregados.	Em serviços/controllers externos aos dados.
Consistência	Garantida pela Raiz de Agregação.	Depende de disciplina manual em cada serviço.
Vocabulário	Linguagem Ubíqua alinhada ao negócio.	Nomes genéricos de tabelas e colunas (id, status, tipo).
Complexidade inicial	Maior; exige modelagem cuidadosa.	Menor; rápido para domínios simples.
Manutenção em domínios complexos	Mais sustentável a longo prazo.	Tende a acumular lógica duplicada e espalhada.
Melhor cenário de uso	Domínios com regras de negócio complexas.	Cadastros simples, sem muita regra de negócio.

Conclusão da comparação: a modelagem anêmica é mais rápida para telas simples de cadastro, mas tende a espalhar regras de negócio por múltiplos serviços à medida que o sistema cresce. O DDD exige mais investimento inicial, mas concentra as regras onde elas pertencem — dentro do próprio modelo de domínio.

5. Onde o DDD foi aplicado seletivamente no Scrunic

Módulo	Abordagem escolhida	Justificativa
Quadro / Colunas / Cartões	DDD com Agregado completo	Regras de consistência complexas entre colunas e cartões
Cadastro de usuário	Modelagem anêmica (CRUD)	Poucas regras de negócio; overhead do DDD não se paga

6. Conclusão

O DDD se mostrou valioso para manter o modelo do Scrunic fiel ao vocabulário do negócio e para proteger as regras de consistência do quadro de tarefas. Sua principal limitação — a curva de aprendizado e o overhead em domínios simples — reforça que ele rende mais quando aplicado seletivamente, nos módulos onde a complexidade de negócio realmente justifica o investimento em um modelo rico.